

Artificial Intelligence (ENCT 351)

Formulas, Algorithms, Derivations & Logical Facts

Compiled from Course Syllabus & Micro-syllabus

Scope of this document. This compendium extracts *only* the mathematical formulas, algorithms, numerical procedures, derivations, and logical/technical facts implied by the syllabus and micro-syllabus. Pure definitions, historical notes, and descriptive “short-note” material are deliberately omitted; where a definition is unavoidable for a formula to make sense, it is stated in one line only.

Contents

1	Problem Solving and Search	2
1.1	Formal Problem Definition	2
1.2	Uninformed Search — Complexity	2
1.3	Informed (Heuristic) Search	2
1.4	Adversarial Search	3
1.5	Local Search and Optimization	3
1.6	Evolutionary Optimization — Genetic Algorithm	4
2	Constraint Satisfaction Problems (CSP)	4
3	Knowledge Representation & Logic	5
3.1	Propositional & Predicate Logic	5
3.2	Bayes’ Theorem & Probabilistic Reasoning	5
3.3	Fuzzy Logic	6
4	Machine Learning Fundamentals	7
4.1	Mathematics for ML	7
4.2	Optimization	7
4.3	MLE, MAP and Cross-Entropy	8
4.4	Bias–Variance & Model Capacity	8
4.5	Decision Tree	8
4.6	Support Vector Machine (SVM)	9
4.7	t-SNE (Unsupervised — Dimensionality Reduction)	9
4.8	Reinforcement Learning	9
4.9	Model Evaluation — Confusion Matrix Metrics	10
5	Neural Networks and Deep Learning	10
5.1	Perceptron	10
5.2	Activation Functions	10
5.3	Backpropagation (Derivation)	11

6 Quick-Reference Summary Table

11

1 Problem Solving and Search

1.1 Formal Problem Definition

Key fact(s): A search problem is the tuple $P = (S, s_0, \text{ACTIONS}, \text{RESULT}, \text{GOAL-TEST}, \text{STEP-COST})$, where S is the state space, s_0 the initial state, $\text{RESULT}(s, a)$ the transition function, and path cost $g(n) = \sum \text{step-cost}$.

- b = branching factor (max children per node), d = depth of shallowest goal, m = maximum depth of the search tree.

1.2 Uninformed Search — Complexity

When to use: Use when no domain-specific heuristic is available; only the problem definition is known.

Algorithm	Complete?	Time	Space	Optimal?
BFS	Yes (b finite)	$O(b^d)$	$O(b^d)$	Yes (unit cost)
DFS	No (infinite space)	$O(b^m)$	$O(bm)$	No
Depth-Limited (ℓ)	No	$O(b^\ell)$	$O(b\ell)$	No
Iterative Deepening (IDS)	Yes	$O(b^d)$	$O(bd)$	Yes (unit cost)

How to use / procedure: BFS uses a FIFO queue (expand shallowest node first); DFS uses a LIFO stack; IDS repeats DFS with depth limit $\ell = 0, 1, 2, \dots$ until a goal is found.

1.3 Informed (Heuristic) Search

Evaluation functions

Greedy Best-First Search: $f(n) = h(n)$

A* Search: $f(n) = g(n) + h(n)$

where $g(n)$ = actual cost from start to n , $h(n)$ = estimated (heuristic) cost from n to goal.

Key fact(s):

- **Admissibility:** $h(n) \leq h^*(n)$ for all n (h^* = true optimal cost to goal). h never overestimates.
- **Consistency (monotonicity):** $h(n) \leq c(n, a, n') + h(n')$ for every edge $n \rightarrow n'$.
- Consistent $\Rightarrow$ Admissible (not conversely).
- A* is *optimal* with an admissible heuristic (tree search) or a consistent heuristic (graph search).
- Greedy best-first is generally *not* optimal and not complete (can loop); A* is generally preferred because it balances actual cost and estimated remaining cost, avoiding greedy's short-sightedness.

How to use / procedure: To solve by hand: (1) tabulate $h(n)$ for every node from the given heuristic table; (2) maintain a frontier ordered by $f(n) = g(n) + h(n)$; (3) at each step expand the node with smallest f ; (4) update g of successors as cumulative path cost from start; (5) stop when the goal node is *expanded* (not merely generated); (6) if two nodes tie on f , break tie using smaller h or arrival order as specified.

1.4 Adversarial Search

Minimax value

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } s \text{ is terminal} \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

How to use / procedure: Build the game tree to terminal states (or a fixed depth with a static evaluation function), back up utility values bottom-up: MAX layers take the maximum of children, MIN layers take the minimum.

Alpha-Beta Pruning

α = best (highest) value found so far for MAX along the path (initially $-\infty$).

β = best (lowest) value found so far for MIN along the path (initially $+\infty$).

Prune when $\alpha \geq \beta$.

Key fact(s):

- At a MAX node: update $\alpha = \max(\alpha, \text{value})$; prune remaining children if $\alpha \geq \beta$.
- At a MIN node: update $\beta = \min(\beta, \text{value})$; prune remaining children if $\alpha \geq \beta$.
- Alpha-beta returns the *same* result as minimax — it only prunes branches that cannot affect the final decision.
- Best case (perfect move ordering) time complexity: $O(b^{d/2})$, effectively doubling the searchable depth compared to plain minimax $O(b^d)$.

1.5 Local Search and Optimization

Hill Climbing

Repeat: move to the neighbour s' with the highest value (steepest-ascent);
Stop when no neighbour has a higher value than the current state.

Key fact(s): Suffers from: local maxima, plateaus (shoulders/flat local maxima), and ridges. It keeps no search tree — only the current node and its neighbours are stored (space $O(1)$ beyond current state).

Simulated Annealing — Acceptance Probability

$$P(\text{accept worse move}) = e^{\Delta E/T}$$

where $\Delta E = \text{Value}(\text{new}) - \text{Value}(\text{current})$ (negative for a worse move in a maximization problem), and T is the current temperature.

How to use / procedure:

1. Pick a random neighbour; compute ΔE .
2. If $\Delta E > 0$ (improvement) — always accept.
3. If $\Delta E \leq 0$ (worse) — accept with probability $e^{\Delta E/T}$; otherwise reject.
4. Decrease T according to a cooling schedule (e.g., $T \leftarrow \alpha T$, $0 < \alpha < 1$) and repeat.

Key fact(s): As $T \rightarrow 0$, the algorithm behaves like hill climbing (almost never accepts worse moves); as $T \rightarrow \infty$, moves are accepted almost randomly. Guaranteed to find the global optimum if T decreases slowly enough.

1.6 Evolutionary Optimization — Genetic Algorithm

Fitness-Proportional (Roulette-Wheel) Selection

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j}$$

where f_i = fitness of chromosome i , N = population size.

How to use / procedure: One GA generation (iteration):

1. **Initialize** population of chromosomes (random bit-strings / encodings).
2. **Evaluate fitness** f_i of every chromosome.
3. **Selection:** choose parents using p_i above (or tournament selection).
4. **Crossover** (probability p_c): pick a crossover point; swap the tail segments of two parent chromosomes to create offspring (single-point crossover).
5. **Mutation** (probability p_m): flip each bit independently with probability p_m .
6. **Replacement:** form the new generation; repeat from step 2 until a termination criterion (max generations / fitness threshold) is met.

2 Constraint Satisfaction Problems (CSP)

Key fact(s): A CSP = (X, D, C) : variables $X = \{X_1, \dots, X_n\}$, domains $D = \{D_1, \dots, D_n\}$, constraints C .

Node Consistency

Variable X_i is node-consistent if every value in D_i satisfies X_i 's unary constraints.
Fix: remove from D_i every value that violates the unary constraint.

Arc Consistency

The arc (X_i, X_j) is consistent iff for *every* value $x \in D_i$ there exists some $y \in D_j$ such that the binary constraint between X_i, X_j is satisfied.
Revise(X_i, X_j): delete x from D_i if no such y exists.

How to use / procedure: AC-3 algorithm: maintain a queue of all arcs; pop an arc (X_i, X_j) , apply *Revise*; if D_i changed (non-empty), re-add all arcs (X_k, X_i) for neighbours X_k of X_i to the queue; if any domain becomes empty $\Rightarrow$ no solution.

Backtracking Search

Depth-first assignment of variables one at a time; after each assignment, check constraints (with optional forward-checking / arc-consistency filtering); if a violation occurs, backtrack to the previous variable and try another value.

Key fact(s): Variable/value ordering heuristics: Minimum Remaining Values (MRV) — pick the variable with fewest legal values left; Degree heuristic — pick the variable involved in most constraints; Least Constraining Value — order values that rule out the fewest choices for neighbours.

3 Knowledge Representation & Logic

3.1 Propositional & Predicate Logic

Key fact(s):

- **Modus Ponens:** from P and $P \rightarrow Q$, infer Q .
- **Modus Tollens:** from $\neg Q$ and $P \rightarrow Q$, infer $\neg P$.
- **Resolution rule:** from $(P \vee Q)$ and $(\neg Q \vee R)$, infer $(P \vee R)$ (resolve on complementary literal $Q, \neg Q$).
- **Conversion to CNF:** (1) eliminate $\leftrightarrow$: $A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$; (2) eliminate $\rightarrow$: $A \rightarrow B \equiv \neg A \vee B$; (3) push $\neg$ inward using De Morgan's laws $\neg(A \wedge B) \equiv \neg A \vee \neg B$, $\neg(A \vee B) \equiv \neg A \wedge \neg B$, $\neg\neg A \equiv A$; (4) distribute $\vee$ over $\wedge$.
- **Predicate logic:** Universal Instantiation — from $\forall x P(x)$ infer $P(a)$ for any constant a ; Existential Instantiation — from $\exists x P(x)$ infer $P(k)$ for a *new* constant k .
- **Unification:** find substitution θ such that $p\theta = q\theta$ (make two expressions identical) — required before resolution in first-order logic.

Forward Chaining

Start from known facts $\rightarrow$ repeatedly fire any rule whose premises are all satisfied $\rightarrow$ add its conclusion to the known facts $\rightarrow$ repeat until goal is derived or no new fact can be added (data-driven).

Backward Chaining

Start from the goal $\rightarrow$ find a rule whose *consequent* matches the goal $\rightarrow$ recursively try to prove each premise of that rule as a sub-goal $\rightarrow$ succeed when all sub-goals reduce to known facts (goal-driven).

How to use / procedure: Worked pattern (as in model question): List all rules symbolically, list given facts, then apply Modus Ponens iteratively (forward chaining) — after each successful match, add the new derived fact to the fact base and re-scan all rules before concluding.

3.2 Bayes' Theorem & Probabilistic Reasoning

Bayes' Theorem

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

Generalized Bayes (Law of Total Probability form)

For a partition $A_1, \dots, A_n$ of the sample space:

$$P(A_i | B) = \frac{P(B | A_i) P(A_i)}{\sum_{j=1}^n P(B | A_j) P(A_j)}$$

Key fact(s):

- Joint probability: $P(A, B) = P(A | B)P(B) = P(B | A)P(A)$.
- Complement rule: $P(\bar{A}) = 1 - P(A)$; useful when a question asks “probability of *not...*” (e.g., $P(\text{not sent to prison}) = 1 - 0.45$).

- Bayesian network joint distribution (chain rule): $P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$.

How to use / procedure: Two-step pattern for exam problems: (a) find the unconditional/-marginal probability requested directly from given percentages; (b) for a conditional-probability-after-evidence question, identify $P(\text{evidence} \mid \text{hypothesis})$ for *each* branch of the partition, compute the denominator as total probability of the evidence, then apply Bayes' formula.

Naïve Bayes Classifier

$$P(C \mid X_1, \dots, X_n) \propto P(C) \prod_{i=1}^n P(X_i \mid C)$$

$$\hat{C} = \arg \max_C P(C) \prod_{i=1}^n P(x_i \mid C)$$

When to use: Use when features can reasonably be assumed *conditionally independent* given the class; works well for discrete/categorical attributes, text classification, and small datasets; fast to train.

How to use / procedure:

1. Compute prior $P(C) = \frac{\text{count}(C)}{N}$ for every class.
2. Compute likelihood $P(x_i \mid C) = \frac{\text{count}(X_i = x_i, C)}{\text{count}(C)}$ for each attribute value from the training table.
3. Multiply prior $\times$ all likelihoods for each class.
4. Predict the class with the larger product (no need to normalize by $P(X)$ since it is common to all classes).
5. **Laplace (add-one) smoothing** if a count is zero: $P(x_i \mid C) = \frac{\text{count}(x_i, C) + 1}{\text{count}(C) + k}$, k = number of possible values of that attribute.

3.3 Fuzzy Logic

Membership Functions

$$\text{Triangular } (a < b < c): \quad \mu(x) = \begin{cases} 0 & x \leq a \\ \frac{x-a}{b-a} & a < x \leq b \\ \frac{b-x}{c-b} & b < x < c \\ 0 & x \geq c \end{cases}$$

$$\text{Trapezoidal } (a < b < c < d): \quad \mu(x) = \begin{cases} 0 & x \leq a \\ \frac{x-a}{b-a} & a < x \leq b \\ 1 & b < x \leq c \\ \frac{d-x}{d-c} & c < x < d \\ 0 & x \geq d \end{cases}$$

$$\text{Gaussian:} \quad \mu(x) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right)$$

Fuzzy Set Operators

Union (OR): $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$ Intersection (AND): $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
 Complement (NOT): $\mu_{A'}(x) = 1 - \mu_A(x)$

How to use / procedure: Fuzzy Inference System (FIS) pipeline: Fuzzification (crisp input $\rightarrow$ membership degree) $\rightarrow$ Rule evaluation (combine antecedents with min/max) $\rightarrow$ Aggregation of rule outputs $\rightarrow$ Defuzzification.

Defuzzification — Centroid Method

$$x^* = \frac{\int x \mu(x) dx}{\int \mu(x) dx}$$

4 Machine Learning Fundamentals

4.1 Mathematics for ML

Key fact(s):

- Linear combination: $v = c_1 v_1 + c_2 v_2 + \dots + c_n v_n$; a **basis** is a minimal spanning, linearly independent set of vectors for a vector space.
- Dot product: $u \cdot v = \sum_i u_i v_i = \|u\| \|v\| \cos \theta$.
- Jacobian (vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$): $J_{ij} = \frac{\partial f_i}{\partial x_j}$ (matrix of first-order partials).
- Hessian (scalar function $f : \mathbb{R}^n \rightarrow \mathbb{R}$): $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ (matrix of second-order partials); used to check convexity ($H \succeq 0$ = positive semi-definite $\Rightarrow$ convex).

4.2 Optimization

Gradient Descent

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

α = learning rate, ∇J = gradient of the cost/loss function.

Key fact(s):

- **Batch GD**: gradient computed over entire dataset per update.
- **Stochastic GD (SGD)**: gradient computed from a single (or mini-batch of) training example per update — noisier but faster per step.
- **Convexity condition**: $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$ for all $\lambda \in [0, 1]$; convex problems have a unique global minimum (no local minima issue), so gradient descent is guaranteed to converge to it.
- **Constrained optimization** typically solved via Lagrange multipliers: minimize $f(x)$ s.t. $g(x) = 0 \Rightarrow \mathcal{L}(x, \lambda) = f(x) + \lambda g(x)$, solve $\nabla \mathcal{L} = 0$.

4.3 MLE, MAP and Cross-Entropy

Maximum Likelihood Estimation (MLE)

$$\theta_{MLE} = \arg \max_{\theta} \prod_i P(x_i | \theta) = \arg \max_{\theta} \sum_i \log P(x_i | \theta)$$

Maximum A Posteriori (MAP)

$$\theta_{MAP} = \arg \max_{\theta} P(\theta | X) = \arg \max_{\theta} \left[\log P(X | \theta) + \log P(\theta) \right]$$

Key fact(s): MAP = MLE + a prior term $\log P(\theta)$; MAP reduces to MLE when the prior is uniform (uninformative).

Cross-Entropy Loss

Binary: $\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$

Multi-class: $\mathcal{L} = -\sum_c y_c \log(\hat{y}_c)$

4.4 Bias–Variance & Model Capacity

Bias–Variance Decomposition

$$\mathbb{E}[(\hat{f}(x) - f(x))^2] = \underbrace{(\text{Bias}[\hat{f}(x)])^2}_{\text{Bias}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{Variance}} + \sigma^2$$

$$\text{Bias}[\hat{f}(x)] = \mathbb{E}[\hat{f}(x)] - f(x)$$

Key fact(s): High bias $\Rightarrow$ underfitting (model too simple); high variance $\Rightarrow$ overfitting (model too complex / too sensitive to training data). Increasing model capacity typically decreases bias but increases variance (bias–variance trade-off).

4.5 Decision Tree

Entropy

$$H(S) = -\sum_i p_i \log_2 p_i$$

Information Gain

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Gini Index

$$Gini(S) = 1 - \sum_i p_i^2$$

How to use / procedure: ID3 / C4.5: at each node, compute IG for every candidate attribute; split on the attribute with *maximum* IG (equivalently minimum weighted entropy). **CART:** split on the attribute/threshold with *minimum* weighted Gini index. Recurse on each branch until pure leaves or stopping criterion (max depth / min samples) is reached.

4.6 Support Vector Machine (SVM)

Hyperplane & Margin

Decision boundary: $w \cdot x + b = 0$ Margin width: $\frac{2}{\|w\|}$

Hard-Margin Optimization

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

Soft-Margin Optimization (with slack ξ_i)

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

C controls the trade-off between margin width and misclassification penalty.

Key fact(s): Kernel trick (for non-linearly separable data): $K(x_i, x_j) = \varphi(x_i) \cdot \varphi(x_j)$ without explicitly computing φ . Common kernels: Linear $K = x_i \cdot x_j$; Polynomial $K = (x_i \cdot x_j + c)^d$; RBF/Gaussian $K = \exp(-\gamma \|x_i - x_j\|^2)$. Only points on/inside the margin (support vectors) determine the boundary.

4.7 t-SNE (Unsupervised — Dimensionality Reduction)

t-SNE similarities

High-dimensional (Gaussian) pairwise similarity: $p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$, symmetrized $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$.

Low-dimensional (Student-t, 1 d.o.f.) similarity: $q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$

Key fact(s): Cost function minimized (via gradient descent on the low-dimensional points y_i): $C = \text{KL}(P \| Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$. The heavy-tailed Student-t distribution in low-D prevents the “crowding problem.”

4.8 Reinforcement Learning

Q-Learning Update Rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

α = learning rate, γ = discount factor, r = immediate reward.

4.9 Model Evaluation — Confusion Matrix Metrics

Term	Formula
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall / Sensitivity	$\frac{TP}{TP + FN}$
Specificity	$\frac{TN}{TN + FP}$
F1-score	$\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

Key fact(s): TP =True Positive, TN =True Negative, FP =False Positive, FN =False Negative. Accuracy alone is misleading on **imbalanced** datasets (e.g., rare-disease/COVID testing) — Precision/Recall/F1 must be checked, especially Recall when missing a positive case (FN) is costly.

5 Neural Networks and Deep Learning

5.1 Perceptron

Perceptron Output & Learning Rule

Output: $o = f(\sum_i w_i x_i + b)$, step activation $f(z) = 1$ if $z \geq 0$, else 0.
 Weight update (if misclassified): $w_i \leftarrow w_i + \eta(t - o)x_i$, $b \leftarrow b + \eta(t - o)$

How to use / procedure: For each training example: compute output o ; if $o \neq t$ (target), update every weight and the bias using the rule above; repeat over the dataset (epochs) until convergence. **Limitation:** a single perceptron can only learn linearly separable functions (e.g., cannot learn XOR) — this motivates the Multilayer Perceptron (MLP).

5.2 Activation Functions

Function	Formula	Derivative
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$
ReLU	$f(z) = \max(0, z)$	1 if $z > 0$, else 0
Softmax	$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	(used at output layer for multi-class)

Key fact(s): Universal Approximation Theorem: a feed-forward network with a single hidden layer containing a finite number of neurons (and a non-linear, non-polynomial activation) can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy, given enough hidden units — it does *not* say how many neurons are needed or how to train it.

5.3 Backpropagation (Derivation)

Key fact(s): Notation for layer l : weighted input $z^l = W^l a^{l-1} + b^l$, activation $a^l = f(z^l)$. Loss example (MSE): $J = \frac{1}{2} \sum (y - a^L)^2$.

Backpropagation Equations

Output-layer error: $\delta^L = \nabla_a J \odot f'(z^L)$

Backpropagated error (hidden layers): $\delta^l = ((W^{l+1})^T \delta^{l+1}) \odot f'(z^l)$

Gradient w.r.t. weights: $\frac{\partial J}{\partial W^l} = \delta^l (a^{l-1})^T$

Gradient w.r.t. bias: $\frac{\partial J}{\partial b^l} = \delta^l$

Weight update: $W^l \leftarrow W^l - \eta \frac{\partial J}{\partial W^l}, \quad b^l \leftarrow b^l - \eta \frac{\partial J}{\partial b^l}$

How to use / procedure: For a 2-layer network (input $\rightarrow$ hidden $\rightarrow$ output), numerically:
 (1) *Forward pass*: compute $z^1 = W^1 x + b^1$, $a^1 = f(z^1)$, $z^2 = W^2 a^1 + b^2$, $a^2 = f(z^2) = \hat{y}$;
 (2) *Compute loss* $J(\hat{y}, y)$; (3) *Backward pass*: compute $\delta^2 = (\hat{y} - y) \odot f'(z^2)$ (for MSE/sigmoid output), then $\delta^1 = (W^2)^T \delta^2 \odot f'(z^1)$; (4) compute gradients $\partial J / \partial W^2 = \delta^2 (a^1)^T$, $\partial J / \partial W^1 = \delta^1 x^T$;
 (5) update every weight/bias using gradient descent; repeat until convergence. This is exactly the chain rule applied backward through the network.

6 Quick-Reference Summary Table

Topic	Core formula(s)
A* search	$f(n) = g(n) + h(n)$; admissible if $h(n) \leq h^*(n)$
Alpha-beta	prune when $\alpha \geq \beta$; best case $O(b^{d/2})$
Simulated annealing	accept worse move with $P = e^{\Delta E/T}$
GA selection	$p_i = f_i / \sum_j f_j$
Bayes' theorem	$P(A B) = P(B A)P(A)/P(B)$
Naive Bayes	$\hat{C} = \arg \max_C P(C) \prod_i P(x_i C)$
Entropy / IG	$H(S) = -\sum p_i \log_2 p_i$; $IG = H(S) - \sum \frac{ S_v }{ S } H(S_v)$
Gini	$1 - \sum p_i^2$
SVM margin	$2/\ w\ $; minimize $\frac{1}{2}\ w\ ^2$
Gradient descent	$\theta \leftarrow \theta - \alpha \nabla J(\theta)$
MLE / MAP	$\arg \max_{\theta} \sum \log P(x_i \theta) / + \log P(\theta)$
Cross-entropy	$-\sum y_c \log \hat{y}_c$
Bias-Variance	Error = Bias ² + Variance + σ^2
Confusion matrix	Accuracy, Precision, Recall, Specificity, F1 (see Sec. 4.8)
Q-learning	$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
Perceptron update	$w_i \leftarrow w_i + \eta(t - o)x_i$
Backpropagation	$\delta^l = ((W^{l+1})^T \delta^{l+1}) \odot f'(z^l)$
Fuzzy operators	$\cup = \max, \cap = \min, \text{NOT} = 1 - \mu$